Deep Comparison & Training Deep Dive — All Three Transformer Architectures

A deep reference covering what's shared, what's different, and how training actually works across Encoder-Only, Decoder-Only, and Encoder-Decoder transformers.

---

17. Deep Comparison — All Three Transformer Architectures

17.1 What's the SAME — The Shared DNA

All three architectures are built from the **same fundamental building blocks**. Think of it like three different houses (bungalow, apartment, duplex) — they all use bricks, cement, wires, and pipes. The arrangement differs, but the materials are identical.

---

1. Embedding + √d_model Scaling

All three convert token IDs → dense vectors using `nn.Embedding`, then scale by `√d_model`.

```
Token ID: 42  →  Embedding lookup  →  [0.12, -0.34, 0.56, ...]  (d_model dims)
                                        × √d_model (e.g., √256 = 16)
                                      → [1.92, -5.44, 8.96, ...]

Why scale? Embedding values are tiny (near 0). Positional encoding values are in [-1, 1].
Without scaling, position would drown out meaning.
```

Architecture	Embedding	Scaling	Difference
Encoder-Only	`nn.Embedding(vocab, d_model)`	`× √d_model`	Single vocab (e.g., English)
Decoder-Only	`nn.Embedding(vocab, d_model)`	`× √d_model`	Single vocab (same language)
Encoder-Decoder	`nn.Embedding(src_vocab, d_model)` + `nn.Embedding(tgt_vocab, d_model)`	`× √d_model` on both	**Two** embeddings (source + target language)

The operation is identical — only the number of embedding tables differs.

---

2. Positional Encoding

All three add position information to embeddings because attention has no built-in sense of order.

```
embedded = Embedding(token_ids) × √d_model
output   = embedded + PositionalEncoding

Position 0: + [sin(0), cos(0), sin(0), cos(0), ...]
Position 1: + [sin(1/10000^0), cos(1/10000^0), sin(1/10000^(2/d)), ...]
Position 2: + [sin(2/10000^0), cos(2/10000^0), ...]
...
```

```
Each position gets a unique wave pattern → model knows word order.
```

Architecture	PE Type	Applied To
Encoder-Only (BERT)	Learned (`nn.Embedding` for positions)	Encoder input
Decoder-Only (GPT-2)	Learned (`nn.Embedding` for positions)	Decoder input
Decoder-Only (LLaMA)	RoPE (rotary, applied to Q and K)	Inside attention
Encoder-Decoder (original)	Sinusoidal (fixed, sin/cos)	Both encoder AND decoder inputs
Our 3 notebooks	Sinusoidal (fixed, sin/cos)	Wherever input enters the model

The concept is the same — the implementation varies. But every transformer needs position information.

---

3. Multi-Head Attention (Q/K/V, Scaled Dot-Product, Softmax)

All three use the exact same attention formula inside every attention layer:

```
Attention(Q, K, V) = softmax(Q × Kᵀ / √d_k) × V
```

The mechanics are identical everywhere:

```
Step 1: Project input into Q, K, V using learned weight matrices
        Q = input × W_Q    K = input × W_K    V = input × W_V

Step 2: Split into n_heads (each head gets d_k = d_model / n_heads dims)

Step 3: Compute attention scores
        scores = Q × Kᵀ / √d_k          ← scaled dot-product

Step 4: Apply softmax → attention weights (sum to 1)

Step 5: Weighted sum of values
        output = weights × V

Step 6: Concatenate all heads → project with W_O
```

What changes is **where Q, K, V come from** and **what mask is applied**:

Attention Type	Q from	K, V from	Mask	Used In
Bidirectional Self-Attention	Same input	Same input	No mask (see all)	Encoder-Only, Enc-Dec encoder
Causal Self-Attention	Same input	Same input	Lower-triangular	Decoder-Only, Enc-Dec decoder
Cross-Attention	Decoder	Encoder output	No mask	Enc-Dec decoder only

But the **formula, the math, the code** inside the attention function is the same in all cases.

---

4. Feed-Forward Network (Expand → Activate → Compress)

All three apply the same FFN independently to each token after attention:

```
FFN(x) = Linear₂(Activation(Linear₁(x)))

Dimensions:
  Linear₁: (d_model → d_ff)     ← expand (typically 4× d_model)
  Activation: ReLU / GELU / SwiGLU
  Linear₂: (d_ff → d_model)     ← compress back

Example with d_model=256, d_ff=1024:
  input: (batch, seq_len, 256)
    → Linear₁ → (batch, seq_len, 1024)    ← expanded
    → ReLU
    → Linear₂ → (batch, seq_len, 256)     ← compressed back
```

Architecture	FFN	Activation	d_ff
Encoder-Only (BERT)	Same	GELU	4 × d_model
Decoder-Only (GPT-2)	Same	GELU	4 × d_model
Decoder-Only (LLaMA)	Same	SwiGLU	~2.67 × d_model
Encoder-Decoder (original)	Same	ReLU	4 × d_model
Our 3 notebooks	Same	ReLU	4 × d_model

The structure is identical — only the activation function varies across models.

---

5. Residual Connections + LayerNorm

All three wrap every sub-layer with a residual (skip) connection and layer normalization:

```
output = LayerNorm(x + sublayer(x))

              x ──────────────────────┐
              │                       │ (skip connection)
              ▼                       │
        Sub-layer (Attention or FFN)  │
              │                       │
              ▼                       │
              + ◄─────────────────────┘  (Add)
              │
              ▼
         LayerNorm                     (Normalize)
              │
              ▼
            output
```

This is applied after EVERY sub-layer in EVERY architecture:

Architecture	Sub-layers per layer	Residual + Norm applied
Encoder-Only	2 (Self-Attn, FFN)	2 times per layer
Decoder-Only	2 (Causal Self-Attn, FFN)	2 times per layer
Encoder-Decoder (encoder)	2 (Self-Attn, FFN)	2 times per layer

Architecture	Sub-layers per layer	Residual + Norm applied
Encoder-Decoder (decoder)	3 (Causal Self-Attn, Cross-Attn, FFN)	3 times per layer

Why residual? Gradients flow directly through the skip connection — even in a 32-layer model, gradients can reach the first layer without vanishing.

Why LayerNorm? Keeps values in a stable range (mean≈0, std≈1) so training doesn't diverge.

---

6. Dropout Regularization

All three apply dropout at the same places:

```
Where dropout is applied (all architectures):
  1. After attention weights (before × V)     ← prevents over-relying on one token
  2. After attention output projection (W_O)   ← regularizes attention output
  3. After FFN                                 ← regularizes FFN output
  4. After positional encoding                 ← regularizes input

Typical rate: 0.1 (10% of values randomly set to 0 during training)
Disabled during inference (model.eval())
```

No difference across architectures — same dropout, same places, same purpose.

---

7. Cross-Entropy Loss

All three use cross-entropy loss — the formula is identical:

```
Loss = -log(P(correct_token))

Model outputs logits: [2.1, 0.5, 8.3, 1.2, ...]   ← one score per vocab word
Softmax → probabilities: [0.01, 0.002, 0.95, 0.005, ...]
If correct token is index 2: Loss = -log(0.95) = 0.05  (low — good prediction!)
If correct token is index 0: Loss = -log(0.01) = 4.6   (high — bad prediction!)
```

What differs is **where** the loss is computed:

Architecture	Loss computed at	Example
Encoder-Only	[CLS] token only (classification)	1 loss value per sentence
Decoder-Only	Every position	6 tokens → 6 loss values → average
Encoder-Decoder	Every target position	5 target tokens → 5 loss values → average

But the loss function itself — `CrossEntropyLoss` — is the same everywhere.

---

8. Adam Optimizer + Gradient Clipping

All three use Adam (or AdamW) and gradient clipping:

```
# This code is IDENTICAL across all three architectures:
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

loss.backward()                                  # compute gradients
```

```
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)  # clip
optimizer.step()                                   # update weights
optimizer.zero_grad()                              # reset for next batch
```

Component	What it does	Same across all?
Adam optimizer	Adaptive learning rate per parameter	✅ Yes
`loss.backward()`	Compute gradients via chain rule	✅ Yes
Gradient clipping	Cap gradient norm to prevent explosion	✅ Yes
`optimizer.step()`	Update weights using gradients	✅ Yes
`optimizer.zero_grad()`	Reset gradients for next iteration	✅ Yes

---

9. Backpropagation (loss.backward → optimizer.step → zero_grad)

All three train using the exact same loop structure:

```
for epoch in range(n_epochs):
    for batch in dataloader:
        # 1. Forward pass (architecture-specific)
        logits = model(batch)

        # 2. Compute loss (same function, different positions)
        loss = criterion(logits, targets)

        # 3. Backward pass (IDENTICAL across all three)
        loss.backward()              ← chain rule through computation graph

        # 4. Clip gradients (IDENTICAL)
        clip_grad_norm_(model.parameters(), max_norm=1.0)

        # 5. Update weights (IDENTICAL)
        optimizer.step()

        # 6. Reset gradients (IDENTICAL)
        optimizer.zero_grad()
```

Steps 3–6 are **literally the same code** regardless of architecture. Only steps 1–2 differ (what the forward pass looks like and where loss is computed).

Summary: The Shared DNA

```
┌──────────────────────────────────────────────────────────────┐
│                     SHARED ACROSS ALL THREE                  │
├──────────────────────────────────────────────────────────────┤
│  ✅ nn.Embedding + √d_model scaling                          │
│  ✅ Positional Encoding (concept — add position to embeddings) │
│  ✅ Multi-Head Attention (Q/K/V, scaled dot-product, softmax) │
│  ✅ Feed-Forward Network (expand → activate → compress)      │
│  ✅ Residual Connections + LayerNorm (after every sub-layer) │
│  ✅ Dropout (attention, FFN, PE — 10%)                       │
│  ✅ Cross-Entropy Loss                                       │
│  ✅ Adam Optimizer + Gradient Clipping                       │
```

```
│  ✅ Backpropagation (backward → clip → step → zero_grad)        │
├──────────────────────────────────────────────────────────────────┤
│  The ARRANGEMENT differs. The BUILDING BLOCKS are the same.     │
└──────────────────────────────────────────────────────────────────┘
```

17.2 What's DIFFERENT — Deep Side-by-Side

Now the interesting part. Same building blocks, but the **arrangement, masking, data flow, and purpose** are fundamentally different.

---

1. Data Format

What does the training data look like before it enters the model?

```
Encoder-Only (BERT):
  Input:  "The movie was great"
  Label:  "positive"                    ← human-provided label
  Format: (text, label) pairs

Decoder-Only (GPT):
  Input:  "the cat sat on the mat"      ← raw text, no labels
  Label:  (comes from the text itself — shifted by 1)
  Format: raw text only

Encoder-Decoder (T5):
  Input:  "I am a student"              ← source sentence
  Target: "je suis un étudiant"         ← target sentence
  Format: (source, target) pairs
```

	Encoder-Only	Decoder-Only	Encoder-Decoder
Training data	(text, label) pairs	Raw text	(source, target) pairs
Labels needed?	✅ Yes (human-annotated)	❌ No (self-supervised)	✅ Yes (paired data)
Data cost	Expensive (labeling)	Cheap (internet text)	Medium (parallel corpora)
Data volume	Thousands–millions	Billions–trillions of tokens	Millions of pairs

---

2. Special Tokens

Each architecture uses different special tokens because they serve different purposes:

```
Encoder-Only:
  [CLS] the movie was great [SEP] [PAD] [PAD]
  │                         │     └─ padding
  │                         └─ sentence boundary
  └─ classification summary token

Decoder-Only:
  the cat sat on the mat
                         │     └─ padding
                         └─ end of sequence (stop generating)

Encoder-Decoder:
```

```
Source: I am a student
Target:  je suis un étudiant
         │                          └─ end of target
         └─ start of target (decoder begins here)
```

Token	Encoder-Only	Decoder-Only	Encoder-Decoder
`[CLS]`	✅ Sentence summary	❌ Not needed	❌ Not needed
`[SEP]`	✅ Sentence boundary	❌ Not needed	❌ Not needed
`[MASK]`	✅ MLM pre-training	❌ Not needed	❌ Not needed
`<sos>` / `<bos>`	❌ Not needed	❌ Not needed (some models use)	✅ Decoder start
`<eos>`	❌ Not needed	✅ Stop signal	✅ End of source & target
`<pad>`	✅ Batch alignment	✅ Batch alignment	✅ Batch alignment
`<unk>`	✅ Unknown words	✅ Unknown words	✅ Unknown words

---

3. Attention Masking — THE Biggest Difference

This is what fundamentally separates the three architectures:

```
Encoder-Only (Bidirectional — NO mask):

  Token:    [CLS]  the  movie  was  great  [SEP]
  [CLS]  [  ✅     ✅    ✅     ✅    ✅     ✅  ]
  the    [  ✅     ✅    ✅     ✅    ✅     ✅  ]
  movie  [  ✅     ✅    ✅     ✅    ✅     ✅  ]   ← EVERY token sees
  was    [  ✅     ✅    ✅     ✅    ✅     ✅  ]     EVERY other token
  great  [  ✅     ✅    ✅     ✅    ✅     ✅  ]
  [SEP]  [  ✅     ✅    ✅     ✅    ✅     ✅  ]

  "great" can see "movie" AND "was" — full context from both sides.


Decoder-Only (Causal — lower-triangular mask):

  Token:    the  cat  sat  on   the  mat
  the    [  ✅   ❌   ❌   ❌   ❌   ❌  ]
  cat    [  ✅   ✅   ❌   ❌   ❌   ❌  ]
  sat    [  ✅   ✅   ✅   ❌   ❌   ❌  ]   ← each token sees only
  on     [  ✅   ✅   ✅   ✅   ❌   ❌  ]     PAST tokens (+ itself)
  the    [  ✅   ✅   ✅   ✅   ✅   ❌  ]
  mat    [  ✅   ✅   ✅   ✅   ✅   ✅  ]

  "sat" can see "the" and "cat" but NOT "on", "the", "mat".


Encoder-Decoder:

  Encoder self-attention: BIDIRECTIONAL (same as encoder-only)
  Decoder self-attention: CAUSAL (same as decoder-only)
  Decoder cross-attention: FULL ACCESS to all encoder outputs (no mask)

  Decoder cross-attention example:
                        Encoder: "I"  "am"  "a"  "student"
```

```
Decoder: ""          [ ✅    ✅    ✅    ✅    ]
Decoder: "je"          [ ✅    ✅    ✅    ✅    ]   ← decoder can see
Decoder: "suis"        [ ✅    ✅    ✅    ✅    ]     ALL encoder tokens
Decoder: "un"          [ ✅    ✅    ✅    ✅    ]     at every step
```

	Encoder-Only	Decoder-Only	Encoder-Decoder
Self-Attention mask	None (bidirectional)	Causal (lower-triangular)	Encoder: none, Decoder: causal
Cross-Attention	❌ None	❌ None	✅ Decoder → Encoder (full access)
Can see future?	✅ Yes (both directions)	❌ No (left-to-right only)	Encoder: yes, Decoder: no

---

4. Number of Sub-layers per Layer

```
Encoder-Only (1 layer):
  ┌──────────────────────────┐
  │ Self-Attention + Add&Norm│  ← sub-layer 1
  │ FFN + Add&Norm           │  ← sub-layer 2
  └──────────────────────────┘
  Total: 2 sub-layers

Decoder-Only (1 layer):
  ┌──────────────────────────┐
  │ CAUSAL Self-Attn + Add&Norm│  ← sub-layer 1
  │ FFN + Add&Norm             │  ← sub-layer 2
  └────────────────────────────┘
  Total: 2 sub-layers

Encoder-Decoder DECODER (1 layer):
  ┌──────────────────────────────┐
  │ CAUSAL Self-Attn + Add&Norm  │  ← sub-layer 1
  │ CROSS-Attn + Add&Norm        │  ← sub-layer 2  ← EXTRA!
  │ FFN + Add&Norm               │  ← sub-layer 3
  └──────────────────────────────┘
  Total: 3 sub-layers (the cross-attention is the extra one)
```

	Encoder-Only	Decoder-Only	Enc-Dec Encoder	Enc-Dec Decoder
Sub-layers	2	2	2	**3**
Self-Attention	✅ Bidirectional	✅ Causal	✅ Bidirectional	✅ Causal
Cross-Attention	❌	❌	❌	✅
FFN	✅	✅	✅	✅

---

5. Output Head — What Sits on Top

The final layer that converts transformer output into something useful:

```
Encoder-Only:
  Encoder output → take [CLS] token → Classification Head → class probabilities
                   (position 0 only)   (Linear: d_model → n_classes)
```

```
  Shape: (batch, seq_len, d_model) → take [:, 0, :] → (batch, d_model) → (batch, n_classes)
  Example: (8, 9, 256) → (8, 256) → (8, 2)  ← 2 classes: positive/negative


Decoder-Only:
  Decoder output → LM Head at EVERY position → next-token probabilities
                   (Linear: d_model → vocab_size)

  Shape: (batch, seq_len, d_model) → (batch, seq_len, vocab_size)
  Example: (1, 6, 256) → (1, 6, 20)  ← 20 vocab words, prediction at every position


Encoder-Decoder:
  Decoder output → LM Head at EVERY decoder position → next-target-token probabilities
                   (Linear: d_model → tgt_vocab_size)

  Shape: (batch, tgt_seq_len, d_model) → (batch, tgt_seq_len, tgt_vocab_size)
  Example: (1, 5, 512) → (1, 5, 8000)  ← 8000 target vocab words
```

	Encoder-Only	Decoder-Only	Encoder-Decoder
Head type	Classification head	LM Head	LM Head
Applied to	[CLS] only (position 0)	Every position	Every decoder position
Output shape	(batch, n_classes)	(batch, seq_len, vocab)	(batch, tgt_len, tgt_vocab)
What it predicts	Class label	Next token	Next target token

---

6. Loss Computation — Where and How

```
Encoder-Only:
  Input:  [CLS] the movie was great [SEP]
  Output: [CLS] embedding → Classification Head → [0.1, 0.9]  (negative, positive)
  Label:  1 (positive)
  Loss:   CrossEntropy([0.1, 0.9], 1) = -log(0.9) = 0.105

  → Loss computed ONCE per sentence, on [CLS] only.


Decoder-Only:
  Input:  [the,  cat,  sat,  on,  the,  mat ]
  Target: [cat,  sat,  on,   the, mat,  ]

  Position 0: model predicts "cat"  → loss₀ = -log(P("cat"))
  Position 1: model predicts "sat"  → loss₁ = -log(P("sat"))
  Position 2: model predicts "on"   → loss₂ = -log(P("on"))
  Position 3: model predicts "the"  → loss₃ = -log(P("the"))
  Position 4: model predicts "mat"  → loss₄ = -log(P("mat"))
  Position 5: model predicts ""→ loss₅ = -log(P(""))

  Total loss = average(loss₀ + loss₁ + ... + loss₅)

  → Loss computed at EVERY position.


Encoder-Decoder:
  Source: [I, am, a, student, ]
```

```
  Target input:  [, je, suis, un, étudiant]
  Target labels: [je, suis, un, étudiant, ]

  Position 0: given      → predict "je"       → loss₀
  Position 1: given je        → predict "suis"     → loss₁
  Position 2: given suis      → predict "un"       → loss₂
  Position 3: given un        → predict "étudiant" → loss₃
  Position 4: given étudiant  → predict ""    → loss₄

  Total loss = average(loss₀ + loss₁ + ... + loss₄)

  → Loss computed at EVERY target position.
```

	Encoder-Only	Decoder-Only	Encoder-Decoder
Loss positions	[CLS] only (1 per sentence)	Every position	Every target position
Loss function	CrossEntropyLoss	CrossEntropyLoss	CrossEntropyLoss
ignore_index	PAD_IDX	PAD_IDX	PAD_IDX
Logits shape	(batch, n_classes)	(batch×seq_len, vocab)	(batch×tgt_len, tgt_vocab)

---

7. Input/Target Creation

How training data is prepared from raw data:

```
Encoder-Only:
  Raw:    "The movie was great" → positive
  Input:  [CLS] the movie was great [SEP] [PAD] [PAD]
  Target: 1 (positive class index)

  → (text, label) pair. Label is external.


Decoder-Only:
  Raw:    "the cat sat on the mat"
  IDs:    [3, 4, 5, 6, 3, 7, 1]           ← tokenized with
  Input:  [3, 4, 5, 6, 3, 7]              ← full_ids[:-1]  (everything except last)
  Target: [4, 5, 6, 3, 7, 1]              ← full_ids[1:]   (everything except first)

  → Shift by 1. Text supervises itself. No external labels.


Encoder-Decoder:
  Raw:    "I am a student" → "je suis un étudiant"
  Source:       [I, am, a, student, ]
  Target input: [, je, suis, un, étudiant]     ← feed to decoder
  Target label: [je, suis, un, étudiant, ]     ← what decoder should predict

  → Source and target are separate. Target is shifted (input has , label has ).
```

	Encoder-Only	Decoder-Only	Encoder-Decoder
Input creation	Add [CLS], [SEP], pad	full_ids[:-1]	Source as-is +
Target creation	External label (int)	full_ids[1:] (shift by 1)	Target shifted: +tokens vs tokens+

	Encoder-Only	Decoder-Only	Encoder-Decoder
Self-supervised?	❌ No (needs labels)	✅ Yes	❌ No (needs pairs)

---

8. Inference Pattern

How each architecture generates output at test time:

```
Encoder-Only — SINGLE PASS:
  Input: [CLS] the movie was great [SEP]
    → Forward pass (one shot)
    → [CLS] output → Classification Head → [0.1, 0.9]
    → argmax → "positive"
  Done. ONE forward pass. No loop.


Decoder-Only — AUTOREGRESSIVE LOOP:
  Prompt: "the cat"
    Step 1: [the, cat]           → model → predicts "sat"
    Step 2: [the, cat, sat]      → model → predicts "on"
    Step 3: [the, cat, sat, on]  → model → predicts "the"
    Step 4: [the, cat, sat, on, the] → model → predicts "mat"
    Step 5: [the, cat, sat, on, the, mat] → model → predicts ""
    STOP.
  N tokens = N forward passes. Sequential, slow.


Encoder-Decoder — ENCODE ONCE + DECODE LOOP:
  Source: "I am a student"
    Encoder: "I am a student" → memory (computed ONCE)

    Decoder loop:
      Step 1: []                          + memory → "je"
      Step 2: [, je]                      + memory → "suis"
      Step 3: [, je, suis]                + memory → "un"
      Step 4: [, je, suis, un]            + memory → "étudiant"
      Step 5: [, je, suis, un, étudiant]  + memory → ""
      STOP.
  1 encoder pass + N decoder passes. Encoder result is reused.
```

	Encoder-Only	Decoder-Only	Encoder-Decoder
Forward passes	**1** (single shot)	**N** (one per token)	**1 + N** (1 encode + N decode)
Loop needed?	❌ No	✅ Yes	✅ Yes (decoder only)
Speed	⚡ Fastest	🐢 Slowest	🐢 Slow (but encoder cached)
Output grows?	No (fixed)	Yes (token by token)	Yes (target grows)

---

9. Weight Tying

Sharing weights between embedding and output head:

```
Weight tying means:
  Embedding matrix:  (vocab_size, d_model)  — converts token ID → vector
```

```
  LM Head matrix:    (d_model, vocab_size)  — converts vector → token scores

  These are transposes of each other! So we share the same matrix:
  lm_head.weight = embedding.weight

  Benefit: fewer parameters, consistent input/output space.
```

	Encoder-Only	Decoder-Only	Encoder-Decoder
Weight tying	❌ No (no LM head in fine-tuning)	✅ Yes (embedding ↔ LM head)	⚠️ Sometimes (target embedding ↔ LM head)
Why/why not	Classification head has different shape	Same vocab in and out	Source and target may have different vocabs
Models that tie	—	GPT-2, LLaMA, Mistral	T5 (ties target emb + LM head)

---

10. Vocabulary — Single vs Dual

```
Encoder-Only:
  ONE vocabulary for ONE language.
  vocab = {"[PAD]": 0, "[CLS]": 1, ..., "the": 5, "cat": 6, ...}
  Size: ~30,000 (BERT)

Decoder-Only:
  ONE vocabulary for ONE language (or multilingual).
  vocab = {"": 0, "": 1, ..., "the": 3, "cat": 4, ...}
  Size: ~50,000 (GPT-2) to ~32,000 (LLaMA)

Encoder-Decoder:
  Can have TWO vocabularies (one per language) or ONE shared vocabulary.

  Option A — Separate vocabs:
    src_vocab = {"": 0, ..., "I": 4, "am": 5, "student": 6}     ← English
    tgt_vocab = {"": 0, ..., "je": 4, "suis": 5, "étudiant": 6} ← French

  Option B — Shared vocab (T5, mBART):
    vocab = {"": 0, ..., "I": 4, "je": 5, "am": 6, "suis": 7, ...}
    Both languages in one vocab. Simpler, enables cross-lingual transfer.
```

	Encoder-Only	Decoder-Only	Encoder-Decoder
Vocabularies	1	1	1 or 2
Embedding tables	1	1	1 (shared) or 2 (separate)
Typical size	~30K	~32K–50K	~32K–50K (shared) or ~10K each

---

11. Training Speed vs Inference Speed

```
Training (all positions computed in parallel):

  Encoder-Only:    FAST  — single forward pass, loss on [CLS] only
  Decoder-Only:    FAST  — single forward pass (causal mask simulates left-to-right)
```

```
  Encoder-Decoder: FAST  — single forward pass through encoder + decoder

  All three are fast during training because teacher forcing allows parallel computation.


Inference:

  Encoder-Only:     ⚡ FASTEST — one forward pass, done
  Decoder-Only:     🐢 SLOW    — N forward passes for N tokens
  Encoder-Decoder:  🐢 SLOW    — 1 + N forward passes

  Generation is inherently sequential — each token depends on the previous one.
```

	Training Speed	Inference Speed	Why
Encoder-Only	⚡ Fast	⚡ Fast	No generation loop, single pass
Decoder-Only	⚡ Fast	🐢 Slow	Autoregressive: N passes for N tokens
Encoder-Decoder	⚡ Fast (slightly slower — 2 components)	🐢 Slow	Encode once + N decode passes

---

Master Comparison Table

Feature	Encoder-Only (BERT)	Decoder-Only (GPT)	Encoder-Decoder (T5)
Purpose	Understanding	Generation	Seq-to-Seq
Data format	(text, label) pairs	Raw text	(source, target) pairs
Special tokens	[CLS], [SEP], [MASK]		,
Attention mask	None (bidirectional)	Causal (lower-triangular)	Bi (encoder) + Causal (decoder) + Cross
Sub-layers/layer	2	2	Encoder: 2, Decoder: 3
Output head	Classification on [CLS]	LM Head at every position	LM Head at every decoder position
Loss positions	[CLS] only	Every position	Every target position
Input/target	(text, label)	shift by 1	(source, shifted target)
Inference	Single pass	Autoregressive loop	Encode once + decode loop
Weight tying	❌ No	✅ Yes	⚠️ Sometimes
Vocabularies	1	1	1 or 2
Training speed	⚡ Fast	⚡ Fast	⚡ Fast
Inference speed	⚡ Fastest	🐢 Slow	🐢 Slow
Examples	BERT, RoBERTa	GPT, LLaMA, Mistral	T5, BART, mBART

17.3 When to Use Which — Decision Flowchart

```
                        What is your task?
                                │
                ┌───────────────┼───────────────┐
                ▼               ▼               ▼
```

```
        UNDERSTAND      GENERATE      TRANSFORM
        (classify,      (write text,   (translate,
         search,         chat,          summarize,
         extract)        complete)      convert)
             │               │               │
             ▼               ▼               ▼
        ┌──────────┐   ┌──────────┐   ┌──────────────┐
        │ ENCODER  │   │ DECODER  │   │ ENCODER-     │
        │ ONLY     │   │ ONLY     │   │ DECODER      │
        │          │   │          │   │              │
        │ BERT     │   │ GPT      │   │ T5, BART     │
        │ RoBERTa  │   │ LLaMA    │   │ mBART        │
        └──────────┘   │ Mistral  │   └──────────────┘
                       └──────────┘
```

Detailed Decision Guide

Your Task	Best Architecture	Why	Example Models
Sentiment analysis	Encoder-Only	Needs full bidirectional context to understand	BERT, RoBERTa
Spam detection	Encoder-Only	Classification = understanding task	BERT
Named Entity Recognition	Encoder-Only	Token-level understanding, needs both sides	BERT
Semantic search	Encoder-Only	Needs sentence embeddings ([CLS])	Sentence-BERT
Text generation	Decoder-Only	Autoregressive = natural for generation	GPT, LLaMA
Chatbot / assistant	Decoder-Only	Conversational generation	ChatGPT, Mistral
Code generation	Decoder-Only	Code is generated left-to-right	Codex, StarCoder
Translation	Encoder-Decoder	Structured input→output mapping	T5, mBART
Summarization	Encoder-Decoder or Decoder-Only	Both work; enc-dec is more structured	BART, GPT-4
Question answering (extractive)	Encoder-Only	Extract span from context	BERT
Question answering (generative)	Decoder-Only	Generate answer from knowledge	GPT, LLaMA
Fill in the blank	Encoder-Only	MLM = bidirectional prediction	BERT

The Modern Reality (2024+)

```
In practice, decoder-only models dominate most tasks now:

  GPT-4, LLaMA, Mistral can do classification, translation, summarization,
  and generation — all through prompting (no architecture change needed).

  "Classify this review as positive or negative: 'Great movie!'"
  → GPT outputs: "positive"
```

```
  But for SPECIALIZED, HIGH-VOLUME tasks:
  - Classification at 500 req/sec → BERT is 10× faster and cheaper
  - Semantic search over millions of docs → Sentence-BERT is purpose-built
  - Structured translation → T5/mBART may give better quality

Rule of thumb:
  - Need to GENERATE text?           → Decoder-Only
  - Need to UNDERSTAND text (fast)?  → Encoder-Only
  - Need structured INPUT → OUTPUT?  → Encoder-Decoder
  - Not sure / want one model?       → Decoder-Only (most flexible)
```

---

18. Deep Dive — Backpropagation & Training Loop

18.1 The Training Loop Anatomy — Step by Step with Shapes

Every transformer trains with the same loop. Let's trace every step with exact tensor shapes.

Setup (using decoder-only as the primary example, d_model=256, vocab=20, seq_len=6, batch=1):

```
model = DecoderOnlyTransformer(...)
criterion = nn.CrossEntropyLoss(ignore_index=PAD_IDX)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
```

Step 1: Forward Pass — Full Data Flow with Tensor Shapes

```
Input token IDs: [3, 4, 5, 6, 3, 7]     ← "the cat sat on the mat"
Shape: (1, 6)                             ← (batch_size, seq_len)

    ┌──────────────────────────────────────────────────────────┐
    │ STEP 1a: Embedding Lookup                                 │
    │   nn.Embedding(20, 256)                                   │
    │   (1, 6) → look up each ID → (1, 6, 256)                  │
    │                                                           │
    │ STEP 1b: Scale by √d_model                                │
    │   (1, 6, 256) × √256 = (1, 6, 256) × 16                   │
    │   → (1, 6, 256)                                           │
    │                                                           │
    │ STEP 1c: Add Positional Encoding                          │
    │   (1, 6, 256) + PE(1, 6, 256) → (1, 6, 256)               │
    │                                                           │
    │ STEP 1d: Dropout                                          │
    │   (1, 6, 256) → randomly zero 10% → (1, 6, 256)           │
    └──────────────────────────────┬────────────────────────────┘
                                   │
                                   ▼
    ┌──────────────────────────────────────────────────────────┐
    │ STEP 1e: Decoder Layer 1 (repeated for each of N layers)  │
    │                                                           │
    │   Causal Self-Attention:                                  │
    │     Q = input × W_Q: (1,6,256) × (256,256) → (1,6,256)    │
    │     K = input × W_K: (1,6,256) × (256,256) → (1,6,256)    │
    │     V = input × W_V: (1,6,256) × (256,256) → (1,6,256)    │
    │                                                           │
    │     Split into 4 heads: (1, 6, 256) → (1, 4, 6, 64)       │
```

```
    │     scores = Q × Kᵀ / √64: (1,4,6,64)×(1,4,64,6) → (1,4,6,6) │
    │     Apply causal mask: upper triangle → -inf                 │
    │     softmax → attention weights: (1, 4, 6, 6)                │
    │     output = weights × V: (1,4,6,6)×(1,4,6,64) → (1,4,6,64)│
    │     Concat heads: (1, 4, 6, 64) → (1, 6, 256)                │
    │     Project: (1,6,256) × W_O(256,256) → (1, 6, 256)          │
    │                                                              │
    │   Residual + LayerNorm:                                      │
    │     LayerNorm(input + attn_output) → (1, 6, 256)             │
    │                                                              │
    │   FFN:                                                       │
    │     Linear₁: (1,6,256) × (256,1024) → (1, 6, 1024)           │
    │     ReLU: (1, 6, 1024) → (1, 6, 1024)                        │
    │     Linear₂: (1,6,1024) × (1024,256) → (1, 6, 256)           │
    │                                                              │
    │   Residual + LayerNorm:                                      │
    │     LayerNorm(input + ffn_output) → (1, 6, 256)              │
    └──────────────────────────────┬──────────────────────────────┘
                                   │
                      (repeat for layers 2, 3, 4)
                                   │
                                   ▼
    ┌──────────────────────────────────────────────────────────────┐
    │ STEP 1f: LM Head                                             │
    │   Linear: (1, 6, 256) × (256, 20) → (1, 6, 20)               │
    │                                                              │
    │   Output: LOGITS — raw scores for each vocab word            │
    │   at each position                                           │
    │                                                              │
    │   Position 0: [2.1, -0.5, 8.3, ...]  ← 20 scores             │
    │   Position 1: [1.4, 7.2, -0.3, ...]  ← 20 scores             │
    │   ...                                                        │
    │   Position 5: [0.8, -1.2, 3.5, ...]  ← 20 scores             │
    └──────────────────────────────┬──────────────────────────────┘
                                   │
                                   ▼
                      logits: (1, 6, 20)
```

Step 2: Loss Computation

```
 Logits:  (1, 6, 20)  ← model's predictions
Targets: [4, 5, 6, 3, 7, 1]  ← "cat sat on the mat "
         shape: (1, 6)

Reshape for CrossEntropyLoss:
  logits:  (1, 6, 20) → (6, 20)    ← flatten batch × seq_len
  targets: (1, 6)     → (6,)       ← flatten

CrossEntropyLoss computes:
  Position 0: logits[0] vs target 4 ("cat")   → loss₀ = -log(softmax(logits[0])[4])
  Position 1: logits[1] vs target 5 ("sat")   → loss₁ = -log(softmax(logits[1])[5])
  Position 2: logits[2] vs target 6 ("on")    → loss₂ = -log(softmax(logits[2])[6])
  Position 3: logits[3] vs target 3 ("the")   → loss₃ = -log(softmax(logits[3])[3])
  Position 4: logits[4] vs target 7 ("mat")   → loss₄ = -log(softmax(logits[4])[7])
  Position 5: logits[5] vs target 1 ("") → loss₅ = -log(softmax(logits[5])[1])
```

```
  loss = average(loss₀ + loss₁ + ... + loss₅)
       = single scalar (e.g., 2.87)
```

Step 3: loss.backward() — What Actually Happens

This is where the magic happens. `loss.backward()` does three things:

```
1. TRAVERSE the computation graph BACKWARDS (from loss to inputs)
2. At each operation, compute the LOCAL GRADIENT using the chain rule
3. ACCUMULATE gradients in each parameter's .grad attribute

The computation graph (simplified):

  input_ids
      │
      ▼
  Embedding ──→ PE ──→ Attention ──→ FFN ──→ ... ──→ LM Head ──→ logits
      │                    │           │                  │           │
      │                    │           │                  │           ▼
      │                    │           │                  │        loss (scalar)
      │                    │           │                  │           │
      │                    │           │                  │     ◀─────┘
      │                    │           │                  │   ∂loss/∂logits
      │                    │           │            ◀─────┘
      │                    │           │          ∂loss/∂W_lm_head
      │                    │     ◀─────┘
      │                    │   ∂loss/∂W_ffn
      │              ◀─────┘
      │            ∂loss/∂W_Q, ∂loss/∂W_K, ∂loss/∂W_V
      ◀──────────────┘
    ∂loss/∂Embedding_weights

After loss.backward():
  Every parameter now has a .grad tensor:
    model.embedding.weight.grad      → (20, 256)    ← how to adjust each word's vector
    model.layers[0].attn.W_Q.grad    → (256, 256)   ← how to adjust query weights
    model.layers[0].ffn.W1.grad      → (256, 1024)  ← how to adjust FFN weights
    model.lm_head.weight.grad        → (256, 20)    ← how to adjust output projection
    ... (every learnable parameter gets a .grad)
```

The Chain Rule in Action:

```
loss depends on logits       → ∂loss/∂logits
logits depend on LM Head     → ∂logits/∂W_lm = decoder_output
LM Head input depends on FFN → ∂decoder_out/∂W_ffn
FFN input depends on Attn    → ∂ffn_in/∂W_attn
...

Chain rule:
  ∂loss/∂W_Q = ∂loss/∂logits × ∂logits/∂decoder_out × ... × ∂attn_out/∂W_Q

Each "×" is a local gradient — PyTorch computes these automatically
by walking backwards through the computation graph.
```

Step 4: Gradient Clipping — Where It Fits

```
After loss.backward() but BEFORE optimizer.step():

  torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)

What it does:
  1. Compute the TOTAL gradient norm across ALL parameters:
     total_norm = √(Σ ||param.grad||²)    for all parameters

  2. If total_norm > max_norm (1.0):
     scale_factor = max_norm / total_norm
     Every param.grad *= scale_factor      ← shrink all gradients proportionally

Example:
  Before clipping:
    W_Q.grad norm = 5.2
    W_K.grad norm = 3.8
    FFN.grad norm = 12.1
    total_norm = √(5.2² + 3.8² + 12.1²) = √(27 + 14.4 + 146.4) = √187.8 = 13.7

  max_norm = 1.0, total_norm = 13.7 > 1.0
  scale_factor = 1.0 / 13.7 = 0.073

  After clipping:
    W_Q.grad norm = 5.2 × 0.073 = 0.38
    W_K.grad norm = 3.8 × 0.073 = 0.28
    FFN.grad norm = 12.1 × 0.073 = 0.88
    total_norm = 1.0  ✓

  Gradients are now safe — no explosion.
```

Step 5: optimizer.step() — How Weights Actually Update

```
Adam optimizer maintains TWO extra values per parameter:
  m = first moment (running average of gradients — "momentum")
  v = second moment (running average of squared gradients — "velocity")

For each parameter W:

  1. Update moments:
     m = β₁ × m + (1 - β₁) × grad           ← momentum (β₁ = 0.9)
     v = β₂ × v + (1 - β₂) × grad²          ← velocity (β₂ = 0.999)

  2. Bias correction (important in early steps):
     m̂ = m / (1 - β₁ᵗ)                       ← t = step number
     v̂ = v / (1 - β₂ᵗ)

  3. Update weight:
     W = W - lr × m̂ / (√v̂ + ε)              ← lr = 0.001, ε = 1e-8

Concrete example for ONE weight value:
  Current weight:  W = 0.5
  Gradient:        grad = 0.02
  Learning rate:   lr = 0.001

  Simple SGD would do:  W = 0.5 - 0.001 × 0.02 = 0.4999998

  Adam does (approximately):
```

```
    m = 0.9 × 0 + 0.1 × 0.02 = 0.002
    v = 0.999 × 0 + 0.001 × 0.0004 = 0.0000004
    W = 0.5 - 0.001 × 0.002 / (√0.0000004 + 1e-8)
    W ≈ 0.5 - 0.001 × 3.16 ≈ 0.4968

  Adam takes a BIGGER step because it adapts to the gradient magnitude.
```

Step 6: optimizer.zero_grad() — Why Needed

```
PyTorch ACCUMULATES gradients by default:

  Iteration 1: loss.backward() → W.grad = 0.02
  Iteration 2: loss.backward() → W.grad = 0.02 + 0.03 = 0.05  ← ACCUMULATED!
  Iteration 3: loss.backward() → W.grad = 0.05 + 0.01 = 0.06  ← keeps growing!

This is WRONG — each iteration should use only its own gradient.

optimizer.zero_grad() resets all .grad to 0 before the next backward pass:

  Iteration 1: zero_grad() → backward() → W.grad = 0.02 → step()
  Iteration 2: zero_grad() → backward() → W.grad = 0.03 → step()  ← clean!
  Iteration 3: zero_grad() → backward() → W.grad = 0.01 → step()  ← clean!

Note: Some advanced techniques (gradient accumulation) intentionally
skip zero_grad() to simulate larger batch sizes. But for normal training,
always zero_grad() before backward().
```

The Complete Training Loop — All Steps Together

```
for epoch in range(n_epochs):
    model.train()                          # enable dropout
    total_loss = 0

    for input_ids, target_ids in dataloader:
        # Step 1: Forward pass
        logits = model(input_ids)          # (batch, seq_len, vocab_size)

        # Step 2: Compute loss
        logits_flat = logits.view(-1, vocab_size)   # (batch*seq, vocab)
        targets_flat = target_ids.view(-1)          # (batch*seq,)
        loss = criterion(logits_flat, targets_flat)  # scalar

        # Step 3: Backward pass (compute gradients)
        loss.backward()

        # Step 4: Clip gradients
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)

        # Step 5: Update weights
        optimizer.step()

        # Step 6: Reset gradients
        optimizer.zero_grad()

        total_loss += loss.item()
```

```
    print(f"Epoch {epoch+1}, Loss: {total_loss / len(dataloader):.4f}")
```

18.2 Backpropagation Through Transformer Layers — How Gradients Flow

Where do gradients go after `loss.backward()`? Let's trace the gradient flow backwards through every component.

The Full Gradient Path (Decoder-Only)

```
loss (scalar)
  │
  ▼
LM Head (Linear: d_model → vocab)
  │
  ▼
Decoder Layer N (last layer)
  ├── FFN
  │    ├── Linear₂ (d_ff → d_model)
  │    ├── ReLU
  │    └── Linear₁ (d_model → d_ff)
  ├── Residual + LayerNorm
  ├── Causal Self-Attention
  │    ├── W_O (output projection)
  │    ├── V weights (W_V)
  │    ├── Attention weights (softmax output)
  │    ├── K weights (W_K)
  │    └── Q weights (W_Q)
  └── Residual + LayerNorm
  │
  ▼
Decoder Layer N-1
  │ (same structure)
  ▼
  ...
  │
  ▼
Decoder Layer 1
  │
  ▼
Positional Encoding (no learnable params — gradients pass through)
  │
  ▼
Embedding (nn.Embedding — lookup table weights get updated)
```

How Gradients Flow Through Residual Connections

This is the **most important** gradient path in the entire transformer:

```
Forward:
  output = LayerNorm(x + sublayer(x))

Backward (gradient flow):
  ∂loss/∂x = ∂loss/∂output × ∂output/∂x
```

```
  Because of the "+" in (x + sublayer(x)):
    ∂output/∂x = 1 + ∂sublayer(x)/∂x
                 ↑
                 This "1" is the KEY!

  The gradient splits into TWO paths:

                    ∂loss/∂output
                         │
                    ┌────┴────┐
                    │         │
              path 1: × 1    path 2: × ∂sublayer/∂x
              (skip conn)    (through sublayer)
                    │         │
                    ▼         ▼
              ∂loss/∂x    ∂loss/∂sublayer_params
              (DIRECT!)   (for W_Q, W_K, etc.)

  Path 1 (skip connection): gradient flows DIRECTLY to the input
    → no multiplication, no shrinking, no vanishing!
    → even in a 32-layer model, gradients reach layer 1 intact

  Path 2 (through sublayer): gradient flows through attention/FFN
    → this path CAN shrink (vanishing) or explode
    → but path 1 always provides a healthy gradient

This is WHY residual connections are essential:
  Without residual: gradient must pass through 32 layers of multiplications → vanishes
  With residual:    gradient has a "highway" that bypasses all layers → always arrives
```

How Gradients Flow Through Attention (Q, K, V Weight Matrices)

```
Forward (simplified for one head):
  Q = input × W_Q          ← (seq, d_model) × (d_model, d_k) → (seq, d_k)
  K = input × W_K
  V = input × W_V
  scores = Q × Kᵀ / √d_k  ← (seq, d_k) × (d_k, seq) → (seq, seq)
  weights = softmax(scores) ← (seq, seq)
  output = weights × V      ← (seq, seq) × (seq, d_k) → (seq, d_k)

Backward — gradient flows through 3 paths:

  ∂loss/∂output
        │
        ├──→ ∂loss/∂V:
        │       ∂loss/∂V = weightsᵀ × ∂loss/∂output
        │       ∂loss/∂W_V = inputᵀ × ∂loss/∂V
        │       → W_V gets updated to produce better "information to share"
        │
        ├──→ ∂loss/∂weights → ∂loss/∂scores (through softmax jacobian):
        │       │
        │       ├──→ ∂loss/∂Q:
        │       │       ∂loss/∂Q = ∂loss/∂scores × K / √d_k
        │       │       ∂loss/∂W_Q = inputᵀ × ∂loss/∂Q
        │       │       → W_Q gets updated to produce better "questions"
        │       │
        │       └──→ ∂loss/∂K:
```

```
      │             ∂loss/∂K = ∂loss/∂scores^T × Q / √d_k
      │             ∂loss/∂W_K = input^T × ∂loss/∂K
      │             → W_K gets updated to produce better "labels/tags"
      │
      └──→ ∂loss/∂W_O (output projection):
             ∂loss/∂W_O = concat_heads^T × ∂loss/∂output
             → W_O gets updated to better combine head outputs

What the model LEARNS through these gradients:
  W_Q learns: "what should each token ask for?"
  W_K learns: "what should each token advertise about itself?"
  W_V learns: "what information should each token share?"
  W_O learns: "how to combine information from all heads?"
```

How Gradients Flow Through FFN

```
Forward:
  hidden = ReLU(input × W₁ + b₁)     ← expand: (seq, 256) → (seq, 1024)
  output = hidden × W₂ + b₂           ← compress: (seq, 1024) → (seq, 256)

Backward:

  ∂loss/∂output (from next layer)
       │
       ▼
  ∂loss/∂W₂ = hidden^T × ∂loss/∂output        ← shape: (1024, 256)
  ∂loss/∂b₂ = sum(∂loss/∂output)               ← shape: (256,)
       │
       ▼
  ∂loss/∂hidden = ∂loss/∂output × W₂^T         ← shape: (seq, 1024)
       │
       ▼
  ∂loss/∂(pre_relu) = ∂loss/∂hidden × ReLU'   ← ReLU' = 1 if x>0, else 0
       │                                          (gradient dies for negative values!)
       ▼
  ∂loss/∂W₁ = input^T × ∂loss/∂(pre_relu)     ← shape: (256, 1024)
  ∂loss/∂b₁ = sum(∂loss/∂(pre_relu))           ← shape: (1024,)

Key insight about ReLU:
  If a neuron's pre-activation was negative → ReLU output was 0
  → gradient through that neuron is also 0 → that neuron doesn't learn this step
  → this is the "dying ReLU" problem (why GELU/SwiGLU are preferred in modern models)
```

How Gradients Reach the Embedding Layer

```
The embedding layer is a lookup table: (vocab_size, d_model)

Forward:
  token ID 4 → look up row 4 → [0.12, -0.34, 0.56, ...]

Backward:
  Only the rows that were USED in this batch get gradients.

  If batch contained tokens [3, 4, 5, 6, 3, 7]:
    Row 3 gets gradient (used at positions 0 and 4 — gradients are SUMMED)
```

```
    Row 4 gets gradient
    Row 5 gets gradient
    Row 6 gets gradient
    Row 7 gets gradient
    Rows 0, 1, 2, 8, 9, ... get ZERO gradient (not used in this batch)

  This is why:
  - Common words (like "the") get updated almost every batch → learn fast
  - Rare words get updated rarely → learn slowly, may stay poorly represented
  -  (row 0) has padding_idx=0 → NEVER gets gradients → stays at zero vector
```

Gradient Flow Summary — Full Picture

```
                    loss
                      │
                      ┌──────────────┐
                      │   LM Head    │  ← ∂loss/∂W_lm_head
                      └──────────────┘
                      │
            ┌─────────┴──────────────┐
            │    Decoder Layer 4     │
            │  ┌──────────────────┐  │
            │  │ FFN              │──┼──→ ∂loss/∂W₁, ∂loss/∂W₂
            │  └──────────────────┘  │
            │     + (residual) ◄─────┼── gradient highway (× 1)
            │  ┌──────────────────┐  │
            │  │ Self-Attention   │──┼──→ ∂loss/∂W_Q, ∂loss/∂W_K, ∂loss/∂W_V, ∂loss/∂W_O
            │  └──────────────────┘  │
            │     + (residual) ◄─────┼── gradient highway (× 1)
            └────────────────────────┘
                      │
            ┌─────────┴──────────────┐
            │    Decoder Layer 3     │  (same gradient paths)
            └────────────────────────┘
                      │
            ┌─────────┴──────────────┐
            │    Decoder Layer 2     │
            └────────────────────────┘
                      │
            ┌─────────┴──────────────┐
            │    Decoder Layer 1     │
            └────────────────────────┘
                      │
                      ┌──────────────┐
                      │  Pos. Enc.   │  ← no learnable params, gradient passes through
                      └──────────────┘
                      │
                      ┌──────────────┐
                      │  Embedding   │  ← ∂loss/∂embedding_rows (only used rows)
                      └──────────────┘

Total gradient paths through a 4-layer model:
  - 4 residual highways (one per layer, each carrying gradient × 1)
  - 4 attention gradient paths (W_Q, W_K, W_V, W_O per layer)
  - 4 FFN gradient paths (W₁, W₂ per layer)
  - 1 LM Head gradient path
  - 1 Embedding gradient path
```

18.3 Training Loop Comparison — All Three Architectures

The backward pass (steps 3–6) is identical. The difference is in the **forward pass** and **loss computation**.

Encoder-Only Training Loop

```
Task: Sentiment classification
Input:  [CLS] the movie was great [SEP]  →  Label: 1 (positive)

┌──────────────────────────────────────────────────────────────────┐
│ Forward Pass:                                                    │
│                                                                  │
│   input_ids: (batch, seq_len) = (8, 9)                          │
│       │                                                          │
│       ▼                                                          │
│   Embedding + PE → (8, 9, 256)                                   │
│       │                                                          │
│       ▼                                                          │
│   Encoder Layer 1 (Bidirectional Self-Attn + FFN) → (8, 9, 256) │
│       │                                                          │
│       ▼                                                          │
│   Encoder Layer 2 → ... → Encoder Layer 4 → (8, 9, 256)         │
│       │                                                          │
│       ▼                                                          │
│   Take [CLS] output: (8, 9, 256) → [:, 0, :] → (8, 256)        │
│       │                                                          │
│       ▼                                                          │
│   Classification Head: (8, 256) → (8, 2)  ← 2 classes           │
│                                                                  │
│ Loss:                                                            │
│   logits: (8, 2)                                                 │
│   labels: (8,)  ← [1, 0, 1, 1, 0, 1, 0, 1]                      │
│   loss = CrossEntropy(logits, labels)  ← ONE loss per sentence   │
│                                                                  │
│ Backward:                                                        │
│   loss.backward()                                                │
│   Gradients flow: Head → [CLS] → all encoder layers → embedding │
│   Note: gradients reach ALL tokens (not just [CLS]) because      │
│   [CLS] attended to all tokens in self-attention                 │
└──────────────────────────────────────────────────────────────────┘
```

Decoder-Only Training Loop

```
Task: Next token prediction (CLM)
Input:  [the, cat, sat, on, the, mat]  →  Target: [cat, sat, on, the, mat, ]

┌──────────────────────────────────────────────────────────────────┐
│ Forward Pass:                                                    │
│                                                                  │
│   input_ids: (batch, seq_len) = (1, 6)                          │
│       │                                                          │
│       ▼                                                          │
│   Embedding + PE → (1, 6, 256)                                   │
│       │                                                          │
```

```
│       ▼                                                          │
│   Decoder Layer 1 (CAUSAL Self-Attn + FFN) → (1, 6, 256)         │
│       │                                                          │
│       ▼                                                          │
│   Decoder Layer 2 → ... → Decoder Layer 4 → (1, 6, 256)          │
│       │                                                          │
│       ▼                                                          │
│   LM Head: (1, 6, 256) → (1, 6, 20)  ← 20 vocab words           │
│                                                                  │
│ Loss:                                                            │
│   logits: (1, 6, 20) → reshape → (6, 20)                         │
│   targets: (1, 6) → reshape → (6,)  ← [4, 5, 6, 3, 7, 1]        │
│   loss = CrossEntropy(logits, targets)  ← loss at EVERY position │
│                                                                  │
│ Backward:                                                        │
│   loss.backward()                                                │
│   Gradients flow: LM Head → all decoder layers → embedding       │
│   Every position contributes to the loss → every position gets   │
│   gradients → very efficient learning (6 training signals from   │
│   1 sentence vs encoder-only's 1 signal)                         │
└──────────────────────────────────────────────────────────────────┘
```

Encoder-Decoder Training Loop

```
Task: Translation
Source: [I, am, a, student, ]
Target input:  [, je, suis, un, étudiant]
Target labels: [je, suis, un, étudiant, ]

┌──────────────────────────────────────────────────────────────────┐
│ Forward Pass:                                                    │
│                                                                  │
│   src_ids: (batch, src_len) = (1, 5)                             │
│   tgt_input_ids: (batch, tgt_len) = (1, 5)                       │
│       │                    │                                     │
│       ▼                    │                                     │
│   ENCODER:                 │                                     │
│   Embedding + PE           │                                     │
│       → (1, 5, 512)        │                                     │
│   Encoder Layers (×6)      │                                     │
│       → (1, 5, 512)        │                                     │
│       = memory             │                                     │
│                            ▼                                     │
│                    DECODER:                                      │
│                    Embedding + PE                                │
│                        → (1, 5, 512)                             │
│                    Decoder Layers (×6):                          │
│                      Causal Self-Attn → (1, 5, 512)              │
│                      Cross-Attn(Q=decoder, K,V=memory)           │
│                        → (1, 5, 512)                             │
│                      FFN → (1, 5, 512)                           │
│                        → (1, 5, 512)                             │
│                    LM Head: (1, 5, 512) → (1, 5, 8000)           │
│                                                                  │
│ Loss:                                                            │
│   logits: (1, 5, 8000) → reshape → (5, 8000)                     │
│   targets: (1, 5) → reshape → (5,)                               │
```

```
│      ← [je_id, suis_id, un_id, étudiant_id, eos_id]              │
│    loss = CrossEntropy(logits, targets)                          │
│      ← loss at every TARGET position                             │
│                                                                  │
│  Backward:                                                       │
│    loss.backward()                                               │
│    Gradients flow TWO paths:                                     │
│      Path 1: LM Head → Decoder layers → Decoder embedding        │
│      Path 2: Decoder cross-attn → Encoder layers → Encoder emb   │
│                                                                  │
│    The encoder gets gradients THROUGH cross-attention!           │
│    Cross-attn uses encoder output as K, V → gradients flow back  │
│    to encoder weights → encoder learns to produce useful memory  │
└──────────────────────────────────────────────────────────────────┘
```

Side-by-Side Summary

Step	Encoder-Only	Decoder-Only	Encoder-Decoder
Forward input	(batch, seq_len)	(batch, seq_len)	src: (batch, src_len) + tgt: (batch, tgt_len)
Forward output	(batch, n_classes)	(batch, seq_len, vocab)	(batch, tgt_len, tgt_vocab)
Loss input	logits: (batch, n_classes)	logits: (batch×seq, vocab)	logits: (batch×tgt_len, tgt_vocab)
Loss target	labels: (batch,)	targets: (batch×seq,)	targets: (batch×tgt_len,)
Loss positions	1 per sentence	All positions	All target positions
Training signals	1 per sentence	seq_len per sentence	tgt_len per sentence
Gradient paths	Head → Encoder → Emb	LM Head → Decoder → Emb	LM Head → Decoder → Encoder → Emb
Backward	`loss.backward()`	`loss.backward()`	`loss.backward()`
Clip	`clip_grad_norm_()`	`clip_grad_norm_()`	`clip_grad_norm_()`
Step	`optimizer.step()`	`optimizer.step()`	`optimizer.step()`
Zero	`optimizer.zero_grad()`	`optimizer.zero_grad()`	`optimizer.zero_grad()`

18.4 Common Training Issues & Fixes

Every transformer, regardless of architecture, can hit these problems. Here's what goes wrong, why, and how to fix it.

Issue 1: Exploding Gradients → Loss becomes NaN

```
Symptom:
  Epoch 1: loss = 3.2
  Epoch 2: loss = 45.7
  Epoch 3: loss = 9482.1
  Epoch 4: loss = NaN        ← model is dead

What happened:
  Gradients grew exponentially large → weight updates were huge →
  weights became extreme → logits became ±inf → softmax produced NaN
```

```
  This often happens in deep models (many layers) where gradients
  multiply through each layer: grad × W × W × W × ... → explosion
```

Fix	How	Code
Gradient clipping	Cap gradient norm	`clip_grad_norm_(params, max_norm=1.0)`
Lower learning rate	Smaller weight updates	`lr=1e-4` instead of `lr=1e-3`
Learning rate warmup	Start with tiny lr, gradually increase	Linear warmup over first 4000 steps
Check data	NaN/inf in input data?	`assert not torch.isnan(input).any()`

Issue 2: Vanishing Gradients → Model doesn't learn

```
Symptom:
  Epoch 1:  loss = 3.2
  Epoch 10: loss = 3.1
  Epoch 50: loss = 3.0
  Epoch 100: loss = 2.99     ← barely improving

What happened:
  Gradients became extremely small as they flowed backwards through layers.
  By the time they reached early layers, they were ~0 → those layers
  didn't update → model couldn't learn deep patterns.

  Common with: very deep models, sigmoid/tanh activations, no residual connections.
```

Fix	How	Why it helps
Residual connections	`output = x + sublayer(x)`	Gradient highway — gradient × 1 through skip connection
LayerNorm	Normalize after each sub-layer	Keeps values in stable range
ReLU/GELU activation	Replace sigmoid/tanh	Gradient = 1 for positive values (doesn't shrink)
Proper initialization	Xavier/Kaiming init	Weights start in a range that preserves gradient magnitude

Transformers already have all of these built in — vanishing gradients are rare in standard transformer architectures.

Issue 3: Overfitting → Training loss drops, validation loss rises

```
Symptom:
  Epoch 1:  train_loss = 3.2,  val_loss = 3.3
  Epoch 10: train_loss = 0.5,  val_loss = 0.6
  Epoch 20: train_loss = 0.1,  val_loss = 0.8    ← gap growing!
  Epoch 30: train_loss = 0.01, val_loss = 1.5    ← memorizing, not learning

What happened:
  The model memorized the training data instead of learning general patterns.
  It performs perfectly on training data but fails on new data.

  Common with: small datasets, large models, too many epochs.
```

Fix	How	Effect
Dropout	`dropout=0.1` (already in transformers)	Randomly disables neurons → forces generalization
Early stopping	Stop when val_loss starts rising	Prevents memorization
More data	Augment or collect more training data	More patterns to learn from
Smaller model	Reduce d_model, n_layers, n_heads	Less capacity to memorize
Weight decay	`AdamW(weight_decay=0.01)`	Penalizes large weights → simpler model
Label smoothing	`label_smoothing=0.1`	Prevents overconfident predictions

Issue 4: Loss not decreasing → Model isn't learning at all

```
Symptom:
  Epoch 1:  loss = 3.2
  Epoch 10: loss = 3.2
  Epoch 50: loss = 3.2       ← completely stuck

What happened:
  Several possible causes:
  1. Learning rate too low → updates are too tiny to make progress
  2. Learning rate too high → model oscillates around minimum, never converges
  3. Bug in data pipeline → model sees garbage data
  4. Bug in loss computation → loss doesn't reflect actual performance
```

Cause	Diagnosis	Fix
LR too low	Loss decreases but extremely slowly	Increase lr (try 10×)
LR too high	Loss oscillates or stays flat	Decrease lr (try 0.1×)
Data bug	Print a batch — do inputs/targets look correct?	Fix data pipeline
Loss bug	Are logits and targets the right shape?	Check shapes before loss
Frozen params	Did you accidentally freeze layers?	Check `param.requires_grad`
Wrong ignore_index	Padding tokens contributing to loss?	Set `ignore_index=PAD_IDX`

Quick debugging checklist:

```
1. Print one batch: input_ids, target_ids — do they make sense?
2. Print logits shape — does it match (batch×seq, vocab)?
3. Print loss value — is it a reasonable number?
4. Print gradient norms — are they non-zero?
   for name, p in model.named_parameters():
       if p.grad is not None:
           print(f"{name}: grad_norm = {p.grad.norm():.4f}")
5. Overfit on ONE batch first — if loss doesn't drop to ~0, there's a bug.
```

Issue 5: Loss is NaN from the start

```
Symptom:
  Epoch 1, Batch 1: loss = NaN    ← immediately broken

What happened:
  1. Input data contains NaN or inf values
  2. Division by zero somewhere (missing ε in normalization)
  3. Extremely large initial weights → logits overflow
  4. Empty batch or zero-length sequence
```

Fix	How
Check data	`assert not torch.isnan(input_ids.float()).any()`
Check for empty sequences	`assert input_ids.shape[1] > 0`
Reduce initial lr	Start with `lr=1e-5`
Use warmup	Gradually increase lr from 0
Check model output	Print `logits.max()`, `logits.min()` — should be reasonable

Training Issues Summary Table

Issue	Symptom	Primary Fix	Secondary Fix
Exploding gradients	Loss → NaN	Gradient clipping	Lower lr
Vanishing gradients	Loss barely decreases	Residual connections	Better activation (GELU)
Overfitting	Train loss ↓, val loss ↑	Dropout + early stopping	More data, smaller model
Not learning	Loss stays flat	Adjust learning rate	Check data pipeline
Immediate NaN	Loss = NaN from batch 1	Check data for NaN/inf	Reduce initial lr
Oscillating loss	Loss jumps up and down	Lower learning rate	Add warmup
Slow convergence	Loss decreases very slowly	Increase learning rate	Use lr scheduler

---

19. Evaluation Metrics — All Three Architectures

Each architecture type has different tasks, so they use different metrics. Here's every major metric with its formula and a simple worked example.

19.1 Encoder-Only Metrics (Classification / Understanding)

Accuracy

```
What: Percentage of correct predictions out of total predictions.

Formula:
  Accuracy = (correct predictions) / (total predictions)
```

```
Example:
  Predictions: [pos, neg, pos, pos, neg, neg, pos, neg, pos, pos]
  Actual:      [pos, neg, pos, neg, neg, pos, pos, neg, pos, neg]
                 ✅    ✅    ✅    ❌    ✅    ❌    ✅    ✅    ✅    ❌

  Correct = 7, Total = 10
  Accuracy = 7/10 = 0.70 = 70%

When to use: Balanced datasets (roughly equal positive/negative).
When NOT to use: Imbalanced datasets — 95% negative → always predicting "negative" gives 95% accuracy!
```

Precision, Recall, F1-Score

```
These handle imbalanced data better than accuracy.

Setup — Confusion Matrix:
                        Predicted
                    Positive    Negative
  Actual Positive    TP=40       FN=10      ← 50 actual positives
  Actual Negative    FP=5        TN=45      ← 50 actual negatives

  TP (True Positive):  predicted positive, actually positive  = 40
  FP (False Positive): predicted positive, actually negative  = 5   ← "false alarm"
  FN (False Negative): predicted negative, actually positive  = 10  ← "missed"
  TN (True Negative):  predicted negative, actually negative  = 45


Precision — "Of everything I predicted positive, how many were actually positive?"

  Formula:  Precision = TP / (TP + FP)
  Example:  Precision = 40 / (40 + 5) = 40/45 = 0.889 = 88.9%

  High precision = few false alarms.
  Important when: false positives are costly (spam filter — don't mark real email as spam).


Recall — "Of everything that was actually positive, how many did I catch?"

  Formula:  Recall = TP / (TP + FN)
  Example:  Recall = 40 / (40 + 10) = 40/50 = 0.80 = 80%

  High recall = few missed positives.
  Important when: false negatives are costly (disease detection — don't miss a sick patient).


F1-Score — "Balance between precision and recall"

  Formula:  F1 = 2 × (Precision × Recall) / (Precision + Recall)
  Example:  F1 = 2 × (0.889 × 0.80) / (0.889 + 0.80)
              = 2 × 0.711 / 1.689
              = 1.422 / 1.689
              = 0.842 = 84.2%

  F1 is the harmonic mean — it penalizes extreme imbalance between precision and recall.
  If precision=0.99 and recall=0.01 → F1=0.02 (terrible, not 0.50).
```

AUC-ROC (Area Under the ROC Curve)

```
What: Measures how well the model separates positive from negative across ALL thresholds.

The model outputs a probability (e.g., 0.73 for positive).
Different thresholds give different precision/recall trade-offs:
  Threshold 0.5: predict positive if P > 0.5
  Threshold 0.3: predict positive if P > 0.3 (catches more positives, more false alarms)
  Threshold 0.8: predict positive if P > 0.8 (fewer false alarms, misses more)

ROC Curve plots: True Positive Rate (recall) vs False Positive Rate at each threshold.
AUC = area under this curve.

  AUC = 1.0  → perfect separation (all positives ranked above all negatives)
  AUC = 0.5  → random guessing (no better than flipping a coin)
  AUC < 0.5  → worse than random (model is confused)

When to use: Binary classification, especially with imbalanced data.
```

Cosine Similarity (for Embeddings)

```
What: Measures similarity between two sentence embeddings.

Formula:
  cos_sim(A, B) = (A · B) / (||A|| × ||B||)

  A · B = sum of element-wise products
  ||A|| = √(sum of squares of A)

Example:
  Sentence 1: "I love this movie"  → [CLS] embedding A = [0.8, 0.6, 0.1]
  Sentence 2: "This film is great" → [CLS] embedding B = [0.7, 0.5, 0.2]

  A · B = 0.8×0.7 + 0.6×0.5 + 0.1×0.2 = 0.56 + 0.30 + 0.02 = 0.88
  ||A|| = √(0.64 + 0.36 + 0.01) = √1.01 = 1.005
  ||B|| = √(0.49 + 0.25 + 0.04) = √0.78 = 0.883

  cos_sim = 0.88 / (1.005 × 0.883) = 0.88 / 0.887 = 0.992

  → Very similar sentences! (close to 1.0)

Range: [-1, 1]
  +1 = identical direction (very similar)
   0 = orthogonal (unrelated)
  -1 = opposite direction (very dissimilar)

Used for: Semantic search, duplicate detection, sentence similarity.
```

19.2 Decoder-Only Metrics (Generation)

Perplexity (PPL)

```
What: "How surprised is the model by the text?" Lower = better.

Formula:
  PPL = e^(average cross-entropy loss)

  Or equivalently:
  PPL = 2^(-1/N × Σ log₂ P(token_i))

  where N = number of tokens, P(token_i) = model's probability for the correct token.

Example:
  Sentence: "the cat sat on the mat"
  Model probabilities for each correct next token:
    P("cat" | "the")           = 0.3
    P("sat" | "the cat")       = 0.5
    P("on" | "the cat sat")    = 0.7
    P("the" | "... sat on")    = 0.4
    P("mat" | "... on the")    = 0.6
    P("" | "... the mat") = 0.8

  Average log probability:
    avg = (log(0.3) + log(0.5) + log(0.7) + log(0.4) + log(0.6) + log(0.8)) / 6
        = (-1.204 + -0.693 + -0.357 + -0.916 + -0.511 + -0.223) / 6
        = -3.904 / 6
        = -0.651

  PPL = e^0.651 = 1.917

  Interpretation: the model is "choosing between ~2 equally likely options" on average.

Benchmarks:
  PPL = 1.0    → perfect (model always predicts the right token with 100% confidence)
  PPL = 10     → decent (choosing between ~10 options)
  PPL = 50     → poor (very uncertain)
  PPL = 1000   → terrible (basically random)

  GPT-2 on WikiText-103: PPL ≈ 29
  GPT-3 on WikiText-103: PPL ≈ 20
  LLaMA-7B:              PPL ≈ 12

Used for: Comparing language models on the same dataset. Lower PPL = better model.
```

BLEU Score (Bilingual Evaluation Understudy)

```
What: Measures overlap between generated text and reference text using n-grams.
Originally for translation, now used for any text generation.

Formula (simplified):
  BLEU = BP × exp(Σ wₙ × log(precisionₙ))

  where:
    precisionₙ = (matching n-grams in generated) / (total n-grams in generated)
    BP = brevity penalty (penalizes too-short outputs)
    wₙ = weight for each n-gram level (usually 1/4 each for n=1,2,3,4)

Example (BLEU-1, unigrams only):
  Generated:  "the cat sat on a mat"
```

```
  Reference:  "the cat sat on the mat"

  Generated unigrams: {the, cat, sat, on, a, mat}  → 6 tokens
  Matching with reference: {the, cat, sat, on, mat} → 5 match ("a" doesn't match)

  BLEU-1 precision = 5/6 = 0.833

Example (BLEU-2, bigrams):
  Generated bigrams:  {the cat, cat sat, sat on, on a, a mat}  → 5 bigrams
  Reference bigrams:  {the cat, cat sat, sat on, on the, the mat}
  Matching: {the cat, cat sat, sat on} → 3 match

  BLEU-2 precision = 3/5 = 0.60

  Final BLEU = BP × exp(0.25×log(0.833) + 0.25×log(0.60) + ...)

Range: 0 to 1 (often shown as 0–100)
  BLEU > 30: understandable translation
  BLEU > 40: high quality
  BLEU > 50: very high quality (close to human)

Limitation: Only measures word overlap — "the cat is happy" and "the feline is joyful"
get a low BLEU score despite meaning the same thing.
```

ROUGE Score (Recall-Oriented Understudy for Gisting Evaluation)

```
 What: Measures overlap focused on RECALL — "how much of the reference is captured?"
Mainly used for summarization.

Variants:
  ROUGE-1: unigram overlap
  ROUGE-2: bigram overlap
  ROUGE-L: longest common subsequence

Formula (ROUGE-1):
  ROUGE-1 Recall    = (matching unigrams) / (total unigrams in REFERENCE)
  ROUGE-1 Precision = (matching unigrams) / (total unigrams in GENERATED)
  ROUGE-1 F1        = 2 × (P × R) / (P + R)

Example:
  Reference: "the cat sat on the mat and slept"  → 8 tokens
  Generated: "the cat sat on the mat"            → 6 tokens

  Matching unigrams: {the, cat, sat, on, the, mat} → 6 match

  ROUGE-1 Recall    = 6/8 = 0.75  ← "generated captures 75% of reference"
  ROUGE-1 Precision = 6/6 = 1.00  ← "everything generated is in reference"
  ROUGE-1 F1        = 2 × (1.0 × 0.75) / (1.0 + 0.75) = 1.5/1.75 = 0.857

ROUGE-L (Longest Common Subsequence):
  Reference: "the cat sat on the mat"
  Generated: "the cat on the mat"

  LCS = "the cat" + "on the mat" = 5 tokens (longest common subsequence, not contiguous)
  ROUGE-L Recall = 5/6 = 0.833

Difference from BLEU:
```

```
  BLEU focuses on PRECISION (is the generated text accurate?)
  ROUGE focuses on RECALL (does the generated text cover the reference?)
```

Human Evaluation Metrics

```
For open-ended generation (chatbots, creative writing), automatic metrics fall short.
Human evaluation is the gold standard:

  Fluency:     "Does the text read naturally?"          (1-5 scale)
  Coherence:   "Does the text make logical sense?"      (1-5 scale)
  Relevance:   "Does the text answer the question?"     (1-5 scale)
  Helpfulness: "Is the response actually useful?"       (1-5 scale)
  Safety:      "Is the response harmful or biased?"     (yes/no)

Used by: ChatGPT (RLHF uses human preferences), LLaMA-Chat, Claude
```

19.3 Encoder-Decoder Metrics (Seq-to-Seq)

Encoder-decoder models use a combination of generation metrics:

BLEU (Translation)

```
Primary metric for machine translation. Same formula as above.

Example:
  Source (English):    "I am a student"
  Reference (French):  "je suis un étudiant"
  Generated (French):  "je suis un élève"

  Unigram matches: {je, suis, un} → 3 out of 4 generated
  BLEU-1 = 3/4 = 0.75

  "élève" is a valid translation of "student" but doesn't match the reference
  → BLEU misses valid synonyms (a known limitation)
```

ROUGE (Summarization)

```
Primary metric for summarization tasks (BART, T5 for summarization).
Same formula as above — measures how much of the reference summary is captured.
```

Exact Match (EM) & F1 (Question Answering)

```
For extractive QA (extract answer span from context):

Exact Match:
  Predicted: "Paris"
  Actual:    "Paris"
  EM = 1 (exact match)
```

```
  Predicted: "the city of Paris"
  Actual:    "Paris"
  EM = 0 (not exact, even though correct)

Token-level F1:
  Predicted tokens: {"the", "city", "of", "Paris"}
  Actual tokens:    {"Paris"}

  Precision = 1/4 = 0.25  (only 1 of 4 predicted tokens is correct)
  Recall    = 1/1 = 1.00  (the actual token "Paris" was found)
  F1        = 2 × (0.25 × 1.0) / (0.25 + 1.0) = 0.50/1.25 = 0.40

  Better prediction: "Paris"
  Precision = 1/1 = 1.0, Recall = 1/1 = 1.0, F1 = 1.0
```

Cross-Entropy Loss / Perplexity (During Training)

```
All three architectures monitor cross-entropy loss during training.
For generation models (decoder-only, encoder-decoder), perplexity is also tracked.

  Training loss going down → model is learning
  Validation loss going down → model is generalizing
  Validation loss going up while training loss goes down → overfitting
```

19.4 Metrics Summary — Which Architecture Uses What

Metric	Encoder-Only	Decoder-Only	Encoder-Decoder	Formula
Accuracy	✅ Primary	❌	❌	correct / total
Precision	✅	❌	❌	TP / (TP + FP)
Recall	✅	❌	❌	TP / (TP + FN)
F1-Score	✅	❌	✅ (QA)	2×P×R / (P+R)
AUC-ROC	✅	❌	❌	Area under TPR vs FPR curve
Cosine Similarity	✅ (embeddings)	❌	❌	A·B / (‖A‖×‖B‖)
Perplexity	❌	✅ Primary	✅	e^(avg cross-entropy)
BLEU	❌	✅	✅ Primary (translation)	n-gram precision + brevity penalty
ROUGE	❌	✅ (summarization)	✅ Primary (summarization)	n-gram recall
Exact Match	❌	❌	✅ (QA)	predicted == actual ? 1 : 0
Human Eval	❌	✅ (chatbots)	❌	Human ratings (1–5)
Cross-Entropy Loss	✅ (training)	✅ (training)	✅ (training)	-log(P(correct))

19.5 Quick Decision: Which Metric for Which Task?

```
┌──────────────────────────────────┬──────────────────────────┐
│ Task                             │ Primary Metric           │
├──────────────────────────────────┼──────────────────────────┤
│ Sentiment classification         │ Accuracy, F1             │
│ Spam detection                   │ Precision, Recall, F1    │
│ Named Entity Recognition         │ F1 (per entity type)     │
│ Semantic search                  │ Cosine Similarity        │
│ Language modeling                │ Perplexity               │
│ Text generation (open-ended)     │ Human Eval, Perplexity   │
│ Machine translation              │ BLEU                     │
│ Summarization                    │ ROUGE                    │
│ Question answering (extractive)  │ Exact Match, F1          │
│ Question answering (generative)  │ ROUGE, Human Eval        │
│ Chatbot / assistant              │ Human Eval               │
└──────────────────────────────────┴──────────────────────────┘
```

---

20. Attention Mechanism — The Complete Story

This section explains attention from scratch — the full story, the "why" behind every piece, and the difference between attention and self-attention. By the end, you should be able to close your eyes and explain it to someone.

20.1 The Problem Attention Solves

Before attention, models processed words one by one (RNNs). The problem:

```
Sentence: "The cat, which was very fluffy and loved to sleep on the couch, sat on the mat."

When the model reaches "sat", it needs to know WHO sat.
The answer is "cat" — but "cat" is 14 words back.

RNN approach:
  Process word by word → by the time you reach "sat",
  the information about "cat" has faded through 14 steps of processing.
  The model forgets.

Attention approach:
  When processing "sat", DIRECTLY look back at every word in the sentence.
  "cat" is right there — no fading, no forgetting.
  "sat" asks: "who is doing the sitting?" and finds "cat" immediately.
```

Attention is a mechanism that lets any word **directly look at any other word**, no matter how far apart they are.

20.2 The Library Story — Understanding Q, K, V from Scratch

Imagine you walk into a **massive library** with 10,000 books. You need information about "cooking Italian pasta."

```
YOU have a QUESTION in your head:
  "I need information about cooking Italian pasta"
  → This is your QUERY (Q)
```

```
Each BOOK on the shelf has a TITLE on its spine:
  Book 1: "Italian Cuisine: A Complete Guide"
  Book 2: "History of Ancient Rome"
  Book 3: "Pasta Making for Beginners"
  Book 4: "Japanese Garden Design"
  Book 5: "Mediterranean Cooking Secrets"
  → These titles are the KEYS (K)

Each book has CONTENT inside:
  Book 1 content: [recipes, techniques, ingredients, ...]
  Book 2 content: [dates, emperors, wars, ...]
  Book 3 content: [dough recipes, sauce tips, ...]
  Book 4 content: [plants, stones, water features, ...]
  Book 5 content: [olive oil tips, herb combinations, ...]
  → These contents are the VALUES (V)
```

Now here's what happens step by step:

```
Step 1: COMPARE your Query against every Key (title)
  Q("cooking Italian pasta") vs K("Italian Cuisine")     → HIGH match! (0.85)
  Q("cooking Italian pasta") vs K("History of Rome")     → LOW match  (0.10)
  Q("cooking Italian pasta") vs K("Pasta Making")        → HIGH match! (0.80)
  Q("cooking Italian pasta") vs K("Japanese Garden")     → NO match   (0.02)
  Q("cooking Italian pasta") vs K("Mediterranean Cook")  → MEDIUM     (0.50)

Step 2: SOFTMAX — turn these scores into percentages (must sum to 100%)
  Book 1: 37%
  Book 2: 5%
  Book 3: 35%
  Book 4: 1%
  Book 5: 22%

Step 3: GRAB the Values (content), weighted by these percentages
  output = 37% × Book1_content + 5% × Book2_content + 35% × Book3_content
         + 1% × Book4_content + 22% × Book5_content

  Result: a rich blend of information, mostly from Book 1 and Book 3,
  with a sprinkle from Book 5. Book 4 is basically ignored.
```

That's attention. Query asks a question. Keys are compared against the query. Values provide the actual information. The output is a weighted mix of values, where the weights come from how well each key matched the query.

20.3 Why THREE Separate Things (Q, K, V)? Why Not Just One?

This is the question everyone asks. Let's think about it:

```
Why not just compare words directly?

  Imagine the word "bank":
    - In "river bank" → it means the side of a river
    - In "bank account" → it means a financial institution

  If we just compared the raw word "bank" with other words,
  we'd get the same comparison every time — regardless of context.

  But what "bank" is LOOKING FOR (Query) depends on context:
    - In "I deposited money at the bank" → Q("bank") asks: "what financial action?"
```

```
    - In "I sat on the river bank" → Q("bank") asks: "what natural feature?"

  And what "bank" ADVERTISES about itself (Key) also depends on context:
    - K("bank") in financial context → "I'm a financial institution"
    - K("bank") in nature context → "I'm a geographical feature"

  And the INFORMATION "bank" provides (Value) is different too:
    - V("bank") in financial context → financial-related features
    - V("bank") in nature context → nature-related features
```

So we need three separate transformations because each serves a **different purpose**:

```
 ┌──────────────────────────────────────────────────────────────────────┐
│                                                                        │
│  Q (Query) — "What am I LOOKING FOR?"                                  │
│    Created by: input × W_Q                                             │
│    Purpose: Represents what this token NEEDS from other tokens         │
│    Analogy: Your search query on Google                                │
│                                                                        │
│  K (Key) — "What do I HAVE TO OFFER?"                                  │
│    Created by: input × W_K                                             │
│    Purpose: Represents what this token CAN PROVIDE to others           │
│    Analogy: The title/tag on a webpage that Google indexes             │
│                                                                        │
│  V (Value) — "Here is my ACTUAL INFORMATION"                           │
│    Created by: input × W_V                                             │
│    Purpose: The actual content that gets passed along                  │
│    Analogy: The actual content of the webpage you read                 │
│                                                                        │
│  Why separate?                                                         │
│    - What you're looking for (Q) ≠ what you advertise (K)              │
│    - What you advertise (K) ≠ what you actually contain (V)            │
│    - Separating them gives the model FLEXIBILITY to learn              │
│      different representations for different purposes                  │
│                                                                        │
└──────────────────────────────────────────────────────────────────────┘
```

Concrete example with a sentence:

```
Sentence: "The cat sat on the mat"

When processing "sat" (position 2):

  Q("sat") = "sat" embedding × W_Q
           = "I'm a verb, I need to find my SUBJECT — who is doing the sitting?"

  Now "sat"'s Query gets compared against every token's Key:

  K("The")  = "The" embedding × W_K = "I'm an article, not very informative"
  K("cat")  = "cat" embedding × W_K = "I'm a noun, I could be a subject"
  K("sat")  = "sat" embedding × W_K = "I'm a verb"
  K("on")   = "on"  embedding × W_K = "I'm a preposition"
  K("the")  = "the" embedding × W_K = "I'm an article"
  K("mat")  = "mat" embedding × W_K = "I'm a noun, I could be an object"

  Score = Q("sat") · K(each token):
    Q("sat") · K("The")  = 0.1   (article — not what "sat" needs)
    Q("sat") · K("cat")  = 0.8   (noun/subject — exactly what "sat" needs!)
```

```
    Q("sat") · K("sat")  = 0.3   (itself — somewhat relevant)
    Q("sat") · K("on")   = 0.2   (preposition — not the subject)
    Q("sat") · K("the")  = 0.1   (article — not relevant)
    Q("sat") · K("mat")  = 0.5   (noun — could be related but not the subject)

  After softmax: [0.05, 0.40, 0.12, 0.08, 0.05, 0.30]

  Now grab the Values:
    V("The")  = "The" embedding × W_V = [actual information about "The"]
    V("cat")  = "cat" embedding × W_V = [actual information about "cat"]
    ...

  output("sat") = 0.05×V("The") + 0.40×V("cat") + 0.12×V("sat")
                + 0.08×V("on") + 0.05×V("the") + 0.30×V("mat")

  Result: "sat" now carries 40% of "cat"'s information and 30% of "mat"'s.
  It now KNOWS that "cat" is the one sitting and "mat" is where.
```

20.4 What Takes What — The Exact Data Flow

Let's trace exactly what goes where, with shapes:

```
 Input: 6 tokens, each represented as a 256-dim vector
X = (6, 256)    ← 6 tokens × 256 dimensions

STEP 1: Create Q, K, V from the SAME input using DIFFERENT weight matrices

  W_Q = (256, 256)    ← learned weights for Query
  W_K = (256, 256)    ← learned weights for Key (DIFFERENT from W_Q!)
  W_V = (256, 256)    ← learned weights for Value (DIFFERENT from both!)

  Q = X × W_Q = (6, 256) × (256, 256) = (6, 256)
  K = X × W_K = (6, 256) × (256, 256) = (6, 256)
  V = X × W_V = (6, 256) × (256, 256) = (6, 256)

  Same input X, but three DIFFERENT projections.
  W_Q, W_K, W_V are all different matrices — they learn different things.


STEP 2: Compute attention scores (how much should each token attend to each other?)

  scores = Q × Kᵀ = (6, 256) × (256, 6) = (6, 6)

  This (6, 6) matrix means:
    Row i, Column j = "how much should token i attend to token j"

    scores = [
      [q₀·k₀, q₀·k₁, q₀·k₂, q₀·k₃, q₀·k₄, q₀·k₅],  ← token 0 vs all
      [q₁·k₀, q₁·k₁, q₁·k₂, q₁·k₃, q₁·k₄, q₁·k₅],  ← token 1 vs all
      [q₂·k₀, q₂·k₁, q₂·k₂, q₂·k₃, q₂·k₄, q₂·k₅],  ← token 2 vs all
      ...
    ]


STEP 3: Scale by √d_k to prevent extreme values

  scores = scores / √256 = scores / 16
```

```
  Why? Without scaling, dot products of 256-dim vectors are huge numbers.
  Huge numbers → softmax becomes nearly one-hot → gradients vanish.


STEP 4: Softmax — convert scores to probabilities (each row sums to 1)

  weights = softmax(scores)    ← shape still (6, 6)

  Each row is now a probability distribution:
    Row 2 (for "sat"): [0.05, 0.40, 0.12, 0.08, 0.05, 0.30]
                         The   cat   sat    on   the   mat
    Sum = 1.0 ✓


STEP 5: Weighted sum of Values

  output = weights × V = (6, 6) × (6, 256) = (6, 256)

  For token 2 ("sat"):
    output[2] = 0.05×V[0] + 0.40×V[1] + 0.12×V[2] + 0.08×V[3] + 0.05×V[4] + 0.30×V[5]

  Each output token is now a BLEND of all other tokens' values,
  weighted by how relevant they are.
```

Summary of what takes what:

```
┌──────────┬──────────────────┬──────────────────────────────────┐
│ Component│ Created from     │ Used for                         │
├──────────┼──────────────────┼──────────────────────────────────┤
│ Q        │ input × W_Q      │ The "question" — compared with K │
│ K        │ input × W_K      │ The "label" — compared with Q    │
│ V        │ input × W_V      │ The "content" — actually passed on │
│ scores   │ Q × Kᵀ           │ Raw similarity between tokens    │
│ weights  │ softmax(scores)  │ How much to attend (probabilities) │
│ output   │ weights × V      │ Context-enriched representation  │
└──────────┴──────────────────┴──────────────────────────────────┘

The KEY insight:
  Q and K determine WHERE to look (the attention pattern).
  V determines WHAT information to actually grab.
  They are decoupled — the "where" and the "what" are independent.
```

20.5 Why Not Just Use Two (Q and V)? Why Do We Need K?

```
You might think: "Why not just compare Q directly with V?"

If Q compared directly with V:
  The same matrix would need to serve TWO purposes:
    1. Be a good "label" for matching (like K does)
    2. Carry useful information to pass along (like V does)

  These are CONFLICTING goals!

  Example: The word "the" appears twice in "The cat sat on the mat"
    - As a label (K): "the" should signal "I'm an article, not important"
```

```
    - As content (V): "the" should carry position/context information

  If K and V were the same, "the" can't be both "unimportant for matching"
  AND "carry useful information" at the same time.

  Separating K and V lets:
    K("the") = "I'm just an article" → low attention score → mostly ignored
    V("the") = [still carries some positional/contextual info if needed]

  The model can learn to IGNORE a token for matching purposes (low K score)
  while still extracting SOME information from it (through V) when needed.
```

20.6 Attention vs Self-Attention — The Difference

This confuses many people. Let's clear it up completely.

Attention is the GENERAL mechanism — the formula:

```
Attention(Q, K, V) = softmax(Q × Kᵀ / √d_k) × V

This formula doesn't care WHERE Q, K, V come from.
They could come from the same sentence, different sentences, or anywhere.
The math is identical regardless.
```

Self-Attention is a SPECIFIC CASE of attention where Q, K, V all come from the **same input**:

```
Self-Attention:
  Input: "The cat sat on the mat"

  Q = this sentence × W_Q    ← from THIS sentence
  K = this sentence × W_K    ← from THIS sentence (same!)
  V = this sentence × W_V    ← from THIS sentence (same!)

  The sentence is looking at ITSELF — hence "self" attention.
  Every word asks: "which OTHER words in MY OWN sentence should I focus on?"
```

Cross-Attention is another specific case where Q comes from one place and K, V come from a **different** place:

```
Cross-Attention (in encoder-decoder):
  Decoder is generating: " je suis"
  Encoder has processed: "I am a student"

  Q = decoder output × W_Q    ← from the DECODER (target sentence)
  K = encoder output × W_K    ← from the ENCODER (source sentence — DIFFERENT!)
  V = encoder output × W_V    ← from the ENCODER (source sentence — DIFFERENT!)

  The decoder is looking at the encoder — hence "cross" attention.
  Each target word asks: "which SOURCE words should I translate from?"
```

Visual comparison:

```
SELF-ATTENTION (sentence looks at itself):

  "The   cat   sat   on   the   mat"
    ↕     ↕     ↕     ↕     ↕     ↕      ← Q, K, V all from here
```

```
    └────┴────┴────┴────┴────┘
    Every word attends to every other word in the SAME sentence


CROSS-ATTENTION (one sentence looks at another):

  Encoder: "I    am   a    student"
             ↑    ↑    ↑    ↑          ← K, V come from here
             |    |    |    |
  Decoder: " je  suis  un"
             ↓    ↓    ↓    ↓          ← Q comes from here
             └────┴────┴────┘
    Each decoder word attends to encoder words (DIFFERENT sentence)
```

Where each type is used:

```
┌──────────────────────┬──────────────────────┬──────────────────────┐
│                      │ Q, K, V from         │ Used in              │
├──────────────────────┼──────────────────────┼──────────────────────┤
│ Self-Attention       │ ALL from same input  │ Encoder-only (BERT)  │
│ (bidirectional)      │                      │ Enc-dec encoder      │
├──────────────────────┼──────────────────────┼──────────────────────┤
│ Causal Self-Attn     │ ALL from same input  │ Decoder-only (GPT)   │
│ (masked, left-only)  │ + causal mask        │ Enc-dec decoder      │
├──────────────────────┼──────────────────────┼──────────────────────┤
│ Cross-Attention      │ Q from decoder       │ Enc-dec decoder ONLY │
│                      │ K, V from encoder    │                      │
└──────────────────────┴──────────────────────┴──────────────────────┘

The FORMULA is the same in all three cases:
  Attention(Q, K, V) = softmax(Q × Kᵀ / √d_k) × V

The ONLY differences are:
  1. Where Q, K, V come from (same input vs different inputs)
  2. Whether a mask is applied (causal mask blocks future tokens)
```

20.7 A Complete Example — Tracing Attention End to End

Let's trace through a tiny example with real numbers.

```
Sentence: "I love cats"    (3 tokens)
d_model = 4 (tiny, for illustration)

Step 0: Embeddings (after lookup + scaling + PE)
  "I"    = [1.0, 0.0, 1.0, 0.0]
  "love" = [0.0, 1.0, 0.0, 1.0]
  "cats" = [1.0, 1.0, 0.0, 0.0]

  X = [[1.0, 0.0, 1.0, 0.0],
       [0.0, 1.0, 0.0, 1.0],
       [1.0, 1.0, 0.0, 0.0]]     shape: (3, 4)


Step 1: Create Q, K, V (using simplified weight matrices)

  W_Q = [[1, 0, 0, 0],           W_K = [[0, 1, 0, 0],           W_V = [[0, 0, 1, 0],
         [0, 1, 0, 0],                  [1, 0, 0, 0],                  [0, 0, 0, 1],
```

```
        [0, 0, 1, 0],                  [0, 0, 0, 1],                  [1, 0, 0, 0],
        [0, 0, 0, 1]]                  [0, 0, 1, 0]]                  [0, 1, 0, 0]]

  Q = X × W_Q:
    Q("I")    = [1.0, 0.0, 1.0, 0.0]
    Q("love") = [0.0, 1.0, 0.0, 1.0]
    Q("cats") = [1.0, 1.0, 0.0, 0.0]

  K = X × W_K:
    K("I")    = [0.0, 1.0, 0.0, 1.0]
    K("love") = [1.0, 0.0, 1.0, 0.0]
    K("cats") = [1.0, 1.0, 0.0, 0.0]

  V = X × W_V:
    V("I")    = [1.0, 0.0, 1.0, 0.0]
    V("love") = [0.0, 1.0, 0.0, 1.0]
    V("cats") = [0.0, 0.0, 1.0, 1.0]


Step 2: Compute scores = Q × Kᵀ

  scores[i][j] = dot product of Q[i] and K[j]

                    K("I")   K("love")  K("cats")
  Q("I")    = [  1×0+0×1+1×0+0×1,  1×1+0×0+1×1+0×0,  1×1+0×1+1×0+0×0 ]
            = [       0.0,               2.0,               1.0          ]

  Q("love") = [  0×0+1×1+0×0+1×1,  0×1+1×0+0×1+1×0,  0×1+1×1+0×0+1×0 ]
            = [       2.0,               0.0,               1.0          ]

  Q("cats") = [  1×0+1×1+0×0+0×1,  1×1+1×0+0×1+0×0,  1×1+1×1+0×0+0×0 ]
            = [       1.0,               1.0,               2.0          ]

  scores = [[0.0, 2.0, 1.0],
            [2.0, 0.0, 1.0],
            [1.0, 1.0, 2.0]]


Step 3: Scale by √d_k = √4 = 2

  scaled = [[0.0, 1.0, 0.5],
            [1.0, 0.0, 0.5],
            [0.5, 0.5, 1.0]]


Step 4: Softmax (each row independently)

  Row 0 ("I"):    softmax([0.0, 1.0, 0.5]) = [0.21, 0.57, 0.35] → normalize → [0.19, 0.50, 0.31]
  Row 1 ("love"): softmax([1.0, 0.0, 0.5]) = [0.50, 0.19, 0.31] → normalize → [0.47, 0.18, 0.35]
  Row 2 ("cats"): softmax([0.5, 0.5, 1.0]) = [0.27, 0.27, 0.45]

  (approximate values — the point is the pattern)

  weights ≈ [[0.19, 0.50, 0.31],     ← "I" attends most to "love"
             [0.47, 0.18, 0.35],     ← "love" attends most to "I"
             [0.27, 0.27, 0.45]]     ← "cats" attends most to itself


Step 5: Weighted sum of Values
```

```
output("I") = 0.19 × V("I") + 0.50 × V("love") + 0.31 × V("cats")
            = 0.19 × [1,0,1,0] + 0.50 × [0,1,0,1] + 0.31 × [0,0,1,1]
            = [0.19, 0.0, 0.19, 0.0] + [0.0, 0.50, 0.0, 0.50] + [0.0, 0.0, 0.31, 0.31]
            = [0.19, 0.50, 0.50, 0.81]

"I" now carries mostly information from "love" (50%) — it knows
that "I" is the one who loves. The attention mechanism connected them.


output("love") = 0.47 × V("I") + 0.18 × V("love") + 0.35 × V("cats")
               = [0.47, 0.0, 0.47, 0.0] + [0.0, 0.18, 0.0, 0.18] + [0.0, 0.0, 0.35, 0.35]
               = [0.47, 0.18, 0.82, 0.53]

"love" now carries mostly information from "I" (47%) and "cats" (35%) —
it knows WHO loves and WHAT is loved.
```

20.8 The One-Sentence Summary

```
Attention lets every token ASK a question (Q), every token ADVERTISE what it has (K),
and then GRAB the actual information (V) from the most relevant tokens.

Self-attention = Q, K, V all come from the SAME sentence (looking at yourself).
Cross-attention = Q from one sentence, K and V from a DIFFERENT sentence (looking at someone else).
The formula is identical — only the source of Q, K, V changes.
```